

SISTEMAS INFORMÁTICOS

PROYECTO INTERMODULAR

1ºDAW

Grupo: **Jianing Xu Yang, Orlando Torrealba, Jairo Alexander.**

Índice

1. Introducción.....	2
2. Desarrollo.....	3
2.1 - Entorno de ejecución de aplicaciones Java.....	3
2.2 - Creación de instaladores para aplicaciones Java.....	5
2.3 - Servicios, APIs y comunicación en red.....	7
2.4 - Contenedores con Docker.....	9
2.5 - Aplicaciones frontend con Angular y React.....	11

1. Introducción

Este proyecto tiene como intención explicar cómo una aplicación desarrollada en Java pasa de ser un programa creado por un desarrollador y ejecutarse correctamente en distintos entornos, como un ordenador, una red o un servidor. Ya que el proyecto se centra en el ámbito de los sistemas informáticos, se documentan aspectos como la instalación, la configuración, la ejecución, los servicios, la seguridad, el despliegue y el mantenimiento básico. También, se analizarán los principales elementos que intervienen en la ejecución de una aplicación, como el sistema operativo, la máquina virtual de Java, los puertos, los servidores, las bases de datos, los permisos, el firewall y las variables de entorno. Por último, la finalidad de este proyecto es comprender y explicar de forma clara y ordenada cómo funcionan los distintos componentes que permiten que una aplicación Java se ejecute correctamente.

2. Desarrollo

2.1 - Entorno de ejecución de aplicaciones Java

- Investiga qué necesita un equipo para ejecutar aplicaciones Java. Explica qué es la JVM y por qué permite ejecutar programas Java en distintos sistemas operativos.

Para ejecutar programas en java, se necesita el JRE el cual incluye la JVM, bibliotecas y componentes necesarios, la JRE significa Java Runtime Environment y son herramientas de software que permite ejecutar aplicaciones basadas en Java en cualquier sistema operativo, teniendo una alta relación con la máquina virtual java (JVM) ya que actúa como intermediario para que todos los programas funcionen correctamente, siendo un traductor y facilitador en el proceso. La JVM como ya mencionamos es una máquina virtual, su función es interpretar y ejecutar el código java (el bytecode) y gracias a esto el mismo programa puede ejecutarse en diferentes sistemas operativos sin necesidad de compilarlo, esto es lo que permite que java sea independiente, indiferentemente de la plataforma que se utilice para compilarlo.

- Diferencia entre JDK y JRE... Indica cuál se necesita para desarrollar y cuál para ejecutar aplicaciones.

Estas son herramientas que ambas comparten y forman parte del entorno de java y están diseñadas para trabajar juntas ya que ambas incluyen la JVM (Java virtual Machine) y permiten correr aplicaciones de java. Sin embargo, a pesar de la gran relación que estas herramientas tienen entre sí, de igual manera tienen sus diferencias que las hacen ser importantes de forma individual, siendo la JRE la herramienta que se encarga únicamente para ejecutar programas de Java en los sistemas operativos, mientras que la JDK (Java Development Kit) es un paquete de software que proporciona las herramientas necesarias para ejecutar y compilar aplicaciones en el lenguaje de Java. Es decir, si solo quieres ejecutar programas necesitas el JRE y si quieres programar en java necesitas el JDK.

- Describe los pasos básicos para instalar Java en Windows o Linux y cómo comprobar la instalación desde la terminal con comandos como `java -version`.

Para instalar Java en Windows o Linux, es muy fácil y podemos hacerlo de formas diferentes en ambos sistemas operativos, los cuales serían: instalador oficial (manual), por el gestor de paquetes del sistema o los por los gestores o herramientas de versiones, todas cumplen con el mismo objetivo de instalar Java en el sistema.

Para instalarlo en linux utilizando la instalación mediante gestor paquetes del sistema se hacen los siguientes pasos:

1. Abrimos la terminal en linux y colocamos el comando: “sudo su” para conectarnos con el usuario root que nos permite trabajar con los permisos de administrador, colocamos la contraseña y escribimos `java --version` para poder verificar la versión de java que tenemos instalada, normalmente aparece **OpenJDK 11.0.3** que es una implementación libre y de código abierto de Java que viene incluida por defecto en muchas distribuciones de Linux, sin embargo aquí instalaremos el JDK oficial de Oracle.
2. Entramos a Google y buscamos “Java JDK” o podemos también entrar directamente a la página web de Oracle, vamos a la opción de “Java Downloads Oracle”, entramos al repositorio de descargas de Java y escogemos Linux, específicamente la opción **x64 Debian Package**, ya que Linux pertenece a la familia Debian.
3. Copiamos el enlace de descarga y descargamos el paquete desde la terminal, escribimos en la terminal el comando “`wget`” y pegamos el enlace a un lado, esto descarga el paquete .deb.
4. Para verificar que se descargó, escribimos el comando “`ls`” y aparecerá el archivo descargado en color rojo. Por ejemplo: “`jdk-22_linux-x64_bin.deb`”.
5. Instalamos el paquete JDK y usamos el comando “`sudo dpkg -i nombre_del_paquete.deb`” y lo que hace es descomprimir, instalar y configurar el Java JDK.
6. Por último verificamos la instalación y escribimos nuevamente “`java - -version`” y debería de aparecer “Java 22” y esto confirma que el nuevo JDK fue instalado correctamente.

Para instalarlo en Windows mediante la instalación manual usando el instalador oficial:

1. Lo primero que tenemos que hacer es buscar en Google “Java Downloads” y entrar a la página de Oracle y entrar en “Java Download | Oracle” donde aparecerá Java 23, Java 21 y versiones anteriores como Java 17, Java 11 o Java 8. En este caso elegimos la versión Java 21, el sistema operativo que sería Windows y el instalador que sería x64 Installer.
2. Abrimos la carpeta de descargas para abrir el instalador y le damos click en ejecutar como administrador y para verificar que ya está instalado, podemos abrir el símbolo del sistema, escribir el comando **cmd** y escribimos `Java - -version` y en caso de que no esté instalado, aparecerá un mensaje indicando que no se reconoce el comando.
3. En el instalador presionamos “next” y escogemos la ruta de instalación que por defecto sería **C:\Program Files\Java\jdk-21** y presionamos nuevamente el “next”, esperamos a que termine de instalarse y presionamos “Close”.

4. Lo verificamos con el terminal escribiendo “Java - -version” y si aparece Java 21, confirma que quedó instalado correctamente.
5. También podemos comprobar que el compilador de Java funciona correctamente escribiendo el comando “javac --version”.
6. Si ambos comandos muestran la versión correspondiente, significa que Java quedó instalado y configurado correctamente en el sistema operativo Windows.

- Explica variables JAVA_HOME y PATH... para qué sirven y qué problemas pueden aparecer si no están configuradas correctamente

Las variables de entorno Java_home y Path son necesarias para que Windows pueda encontrar correctamente la instalación de Java y utilizar sus herramientas sin problemas. La variable Java_Home guarda la ubicación donde está instalado el JDK, mientras que Path permite ejecutar comandos como java o javac desde cualquier ventana del terminal sin tener que escribir toda la ruta de instalación y gracias a esto, programas y entornos de desarrollo como IntelliJ IDEA, Eclipse o Maven pueden detectar Java automáticamente y funcionar de manera correcta.

Si estas variables no están configuradas bien, pueden aparecer varios errores. Por ejemplo, uno de los más comunes es que el sistema muestre un mensaje indicando que el comando java no se reconoce. También pueden surgir problemas al compilar programas, ejecutar proyectos o utilizar herramientas que dependan de Java. En algunos casos, cuando hay varias versiones de Java instaladas, el sistema puede usar una versión equivocada y provocar incompatibilidades o fallos en la ejecución de aplicaciones. Por eso es importante configurar correctamente Java_home y Path después de instalar Java en Windows.

2.2 - Creación de instaladores para aplicaciones Java

- Define qué es un instalador y qué ventajas tiene frente a entregar simplemente un archivo comprimido o un .jar.

Un instalador es un programa que sirve para instalar una aplicación de forma más fácil en el ordenador. Básicamente se encarga de copiar los archivos necesarios, crear carpetas, accesos directos y hacer ciertas configuraciones para que el programa funcione correctamente. En muchos casos también configura automáticamente algunas cosas del sistema, por ejemplo las variables de entorno en Java, para que el usuario no tenga que hacerlo manualmente.

La ventaja que tiene un instalador al entregar solo un archivo comprimido o un .jar es que hace todo mucho más cómodo y simple para la persona que lo va a usar. Con el instalador normalmente solo hay que darle a “next” varias veces y el programa ya queda listo. En cambio, si se entrega un archivo comprimido, el usuario tiene que descomprimirlo y muchas veces no sabe exactamente dónde colocar los archivos o qué hacer después. Con un .jar pasa algo parecido, porque aunque se puede ejecutar directamente, depende de que Java ya esté instalado y configurado en el equipo, y si no lo está puede dar errores o simplemente no abrir.

Además, un instalador suele dar una sensación más completa y profesional, porque puede agregar accesos directos, aparecer en la lista de programas instalados e incluso incluir un desinstalador. Todo eso hace que sea más práctico, sobre todo para usuarios que no tienen mucha experiencia configurando programas manualmente.

- Investiga herramientas que permitan generar instaladores o ejecutables para aplicaciones Java, por ejemplo Inno Setup, install4j, jpackage o Launch4j.

Existen diferentes herramientas que permiten generar instaladores o archivos ejecutables para aplicaciones Java, facilitando su distribución y haciendo que la instalación sea más sencilla para los usuarios. Algunas de las más utilizadas son:

- **Inno Setup:** Es una herramienta bastante conocida para crear instaladores en Windows. Se usa mucho porque permite hacer asistentes de instalación parecidos a los de programas profesionales, creando accesos directos, carpetas y hasta un desinstalador. Además, es gratuita y bastante fácil de usar una vez entiendes cómo funcionan los scripts.
- **Launch4j:** Esta herramienta sirve para convertir un archivo .jar de Java en un archivo .exe. Eso hace que el programa se vea más “normal” para los usuarios de Windows, ya que simplemente hacen doble clic y el programa abre como cualquier otra aplicación. También deja configurar iconos y comprobar si la versión de Java instalada es compatible.
- **install4j:** Es una herramienta más profesional y de pago, usada sobre todo en proyectos grandes. Permite crear instaladores para Windows, Linux y macOS desde una misma aplicación. Tiene una interfaz gráfica bastante cómoda y facilita mucho la configuración de la instalación sin tener que escribir tantas cosas manualmente.
- **jpackage:** Es una herramienta que ya viene incluida en versiones recientes de Java. Sirve para crear instaladores nativos directamente desde el proyecto Java, por ejemplo archivos .exe o .msi en Windows. Una ventaja importante es que puede incluir el propio Java dentro de la aplicación, así el usuario no tiene que instalarlo aparte.
- **IzPack:** Es una herramienta pensada para hacer instaladores multiplataforma para programas Java. Aunque no es tan popular como otras, sigue siendo útil porque permite personalizar bastante el proceso de instalación y funciona en diferentes sistemas operativos.

- **NSIS:** Es otra herramienta bastante utilizada para crear instaladores en Windows. Funciona mediante scripts y permite hacer instaladores ligeros y rápidos. Mucha gente la usa porque da bastante libertad para personalizar cómo se instala el programa.

- Explica qué elementos puede incluir un instalador: archivos de la aplicación, accesos directos, configuración inicial, dependencias, desinstalador y rutas de instalación.

Un instalador puede incluir varias cosas necesarias para que un programa quede funcionando correctamente en el ordenador. Lo más importante son los archivos de la aplicación, es decir, todos los archivos que el programa necesita para poder abrirse y funcionar, como ejecutables, imágenes, librerías o configuraciones internas, además también suele crear accesos directos, por ejemplo en el escritorio o en el menú de inicio, para que el usuario pueda abrir el programa más rápido sin tener que buscar la carpeta donde se instaló. Además, muchos instaladores hacen una configuración inicial automáticamente, dejando algunas opciones ya preparadas para que el programa funcione desde el primer momento.

Otro punto importante son las dependencias, ya que algunas aplicaciones necesitan otros programas o componentes instalados previamente, como Java o ciertas librerías del sistema. El instalador puede comprobar si ya están instalados y en caso de que no, instalarlos automáticamente para evitar errores después. Normalmente también incluyen un desinstalador, que sirve para borrar el programa de forma más sencilla cuando ya no se quiere usar. Así no hace falta eliminar carpetas manualmente ni preocuparse por dejar archivos sueltos en el sistema. Por último, el instalador permite elegir la ruta de instalación, o sea, la carpeta donde se guardará el programa. Aunque casi siempre viene una ruta recomendada por defecto, el usuario puede cambiarla si quiere instalarlo en otro lugar.

2.3 - Servicios, APIs y comunicación en red

- Explica qué es una API y qué papel tiene dentro del modelo cliente-servidor. El objetivo es entender la comunicación entre aplicaciones, no programar una API completa.

Una API es una forma que tienen las aplicaciones de comunicarse entre ellas, es decir, básicamente funciona como un intermediario que recibe una petición, la envía al sistema correspondiente y después devuelve una respuesta. Las siglas API significan Application Programming Interface. Dentro del modelo cliente-servidor, la API tiene el papel de conectar ambas partes, en este caso el cliente es la aplicación o interfaz que utiliza el usuario, mientras que el servidor es el encargado de procesar la información y manejar los datos. Cuando el cliente necesita realizar una acción o solicitar información, envía una petición a través de la API y el servidor responde con los datos o resultados correspondientes.

Las APIs son importantes porque permiten que diferentes sistemas trabajen juntos aunque hayan sido desarrollados con tecnologías distintas. Además ayudan a organizar mejor las aplicaciones, separando la parte visual y la interacción con el usuario de la parte encargada de

procesar datos y realizar operaciones internas y esto hace que el desarrollo sea más ordenado, flexible y más fácil de mantener con el tiempo.

- Investiga qué es Spring Boot y por qué se utiliza para crear servicios o APIs en Java.

Spring Boot es una herramienta basada en el framework Spring que se utiliza para desarrollar aplicaciones y servicios en Java de una forma más rápida y sencilla. Su principal objetivo es facilitar la configuración y reducir la cantidad de código y archivos necesarios para iniciar un proyecto. Antes de Spring Boot, trabajar con Spring podía ser más complicado porque había que configurar muchas cosas manualmente, mientras que con Spring Boot gran parte de esa configuración ya viene preparada automáticamente. Se utiliza mucho para crear APIs y servicios web porque permite desarrollar aplicaciones cliente-servidor de manera más organizada y eficiente. Con Spring Boot es posible crear endpoints, manejar peticiones HTTP y conectar fácilmente la aplicación con bases de datos u otros servicios. Además de eso, incorpora servidores integrados como Tomcat, por lo que no hace falta instalar un servidor externo para ejecutar la aplicación. Otra razón por la que es tan utilizado es porque simplifica bastante el desarrollo de proyectos grandes. Incluye herramientas para gestionar dependencias, seguridad, configuración y pruebas, haciendo que el desarrollo sea más rápido y más fácil de mantener. También tiene una gran comunidad y mucha documentación, por lo que es muy común encontrarlo en aplicaciones empresariales y APIs modernas desarrolladas en Java.

- Describe conceptos básicos de red relacionados con una API: dirección IP, puerto, protocolo HTTP/HTTPS, peticiones GET y POST, y respuesta del servidor.

- **Dirección IP:** Es la dirección que identifica a un dispositivo dentro de una red, ya sea internet o una red local. Funciona de manera parecida a una dirección de casa, ya que permite saber a dónde deben enviarse los datos. Cada dispositivo conectado a una red tiene una dirección IP para poder comunicarse con otros equipos y servidores.
- **Puerto:** Un puerto es como un punto de acceso o canal de comunicación dentro de un dispositivo. Sirve para diferenciar los distintos servicios que pueden estar funcionando al mismo tiempo en un servidor o en un ordenador. Por ejemplo, una API puede estar escuchando peticiones en un puerto específico para recibir solicitudes de los clientes.
- **HTTP y HTTPS:** Son protocolos de comunicación utilizados para transferir información entre el cliente y el servidor. HTTP permite enviar y recibir datos en aplicaciones web o APIs, mientras que HTTPS funciona de la misma forma pero añadiendo una capa de seguridad mediante cifrado. Gracias a HTTPS, la información viaja protegida y es más difícil que terceros puedan interceptarla.
- **Petición GET:** Es un tipo de solicitud que se utiliza principalmente para obtener información desde el servidor. Con una petición GET, el cliente pide datos, pero

normalmente no modifica nada en el sistema y es una de las peticiones más utilizadas en APIs para consultar información.

- **Petición POST:** Este tipo de petición se utiliza para enviar datos al servidor. Generalmente sirve para crear información nueva, guardar registros o enviar formularios. A diferencia de GET, una petición POST sí suele producir cambios en la aplicación o en la base de datos.
- **Respuesta del servidor:** Después de recibir una petición, el servidor procesa la información y devuelve una respuesta al cliente. Esa respuesta puede contener datos solicitados, mensajes indicando que la operación fue correcta o errores en caso de que ocurra algún problema y normalmente las respuestas de una API se envían en formatos como JSON porque son fáciles de interpretar tanto para programas como para personas.

2.4 - Contenedores con Docker

- Explica qué es Docker y qué problema intenta resolver: ejecutar aplicaciones en entornos aislados y reproducibles.

- **Docker** es una plataforma que crea y ejecuta aplicaciones dentro de contenedores (entornos aislados) que tienen dentro lo necesario para que la propia aplicación funcione.
- El problema que **Docker** intenta resolver es que una aplicación pueda funcionar de manera perfecta o como se espera en todo **S.O** sin importar en qué **S.O** se ejecuta, independientemente de sus diferentes características, permitiendo que funcione igual en cualquier entorno de forma aislada y fácil de desplegar.

- Diferencia entre máquina virtual y contenedor. Indica ventajas e inconvenientes de cada enfoque de forma sencilla.

- **Máquina virtual:** Emula un ordenador completo con un **hipervisor**, que permite crear varios S.O dentro de un mismo PC.

VENTAJAS → Puedes hacer **todas** las pruebas que necesites en distintos S.O, si una máquina virtual falla, no afecta al resto.

INCONVENIENTES → Configurar una máquina virtual **consume** RAM, CPU y almacenamiento. Esto quiere decir que al arrancar una máquina virtual, el arranque puede ser **lento**, las imágenes de una máquina virtual puede ocupar mucho espacio.

- **Contenedor:** Ejecuta aplicaciones de manera aislada **compartiendo** el núcleo del mismo S.O, no incluye un S.O completo.

VENTAJAS → En cuanto a espacio ocupa **mucho menos** que una máquina virtual, esto ayuda a que arranque en **poco tiempo**, consume **pocos recursos**.

INCONVENIENTES → **Dependencia completa del kernel**, hay limitaciones entre sistemas. Trabajar en grandes proyectos puede ser complejo ya que gestionar muchos contenedores puede complicarse.

- Define los conceptos básicos: imagen, contenedor, Dockerfile, volumen y puerto expuesto.
 - **Imagen:** Paquete que incluye código, librerías, dependencias y configuración. En resumen es como una “plantilla” de solo lectura con todo lo necesario para ejecutar una aplicación, usado para crear contenedores en Docker.
 - **Contenedor:** Es el entorno donde se ejecuta la aplicación usando una imagen. Comparte una parte del núcleo del S.O host, pero igualmente funciona de forma independiente.
 - **Dockerfile:** Es un archivo de texto con las “instrucciones” para crear una imagen “Docker” (instalar programas, copiar archivos, ejecutar comandos).
 - **Volumen:** Es un mecanismo que tiene Docker para guardar datos de forma persistente fuera del contenedor, así cuando se elimina, no se pierde.
 - **Puerto expuesto:** Es el puerto que un contenedor deja disponible para que haya comunicación con el “exterior” o con otros contenedores. Se suele asociar a servicios web, por ejemplo “HTTP”.
- Relaciona Docker con Java: cómo podría empaquetarse una aplicación Spring Boot en una imagen y ejecutarse en cualquier equipo con Docker instalado.
 - Docker se relaciona bien con Java, se puede empaquetar una aplicación completa y ejecutarla igual en **cualquier máquina**.
 - En el caso de una aplicación **Spring Boot**, se compila en un archivo **.jar** o **.war** y luego se crea la **imagen Docker** que incluye ese archivo con una versión de Java. Esto ayuda a que el contenedor pueda ejecutar la aplicación **sin importar la versión de Java** del equipo.
 - En resumen, una aplicación Spring Boot dentro de Docker se puede ejecutar en cualquier equipo **con Docker instalado**, sin dar errores de compatibilidad.

2.5 - Aplicaciones frontend con Angular y React

- Explica qué es una aplicación frontend y su papel en cliente-servidor.
 - **Aplicación frontend:** es la parte de la aplicación que nosotros **vemos** como usuarios, con la que interactuamos directamente (interfaz gráfica). Está compuesta normalmente con **HTML, CSS y JavaScript**.
 - **Frontend en cliente-servidor:** el frontend hace de cliente que se ejecuta en el navegador, cuando enviamos una petición por ejemplo para guardar información, el servidor responde y el frontend se encarga de **mostrar esos datos de forma visual** y comprensible.
- Investiga Angular y React.
 - **Angular:** Es un **framework** desarrollado por Google. Permite crear aplicaciones web estructuradas de forma organizada. Tiene herramientas integradas como “rutas, formularios, comunicación con servidores”.
 - **React:** Es una **librería** desarrollada por Facebook. Se centra en construir **interfaces de usuario**. Es más flexible que Angular, pero es necesario añadir librerías para funcionalidades extra.
- Describe ejecución: Node.js, npm, servidor dev.
 - **Node.js:** Es un “entorno de ejecución” que permite ejecutar JavaScript **fuera del navegador**, normalmente en el servidor.
 - **Npm:** (Node Package Manager) es el gestor de paquetes de Node.js. Se utiliza para **instalar y gestionar librerías o dependencias** que necesita la aplicación, por ejemplo Angular o React.
 - **Servidor dev:** Es el **servidor local** que se usa **mientras se desarrolla una aplicación**. Ayuda bastante con las pruebas porque se pueden ver los cambios de la app en tiempo real en el propio navegador sin tener que desplegar la app.
- Explica build y archivos generados.
 - **Build:** Es el proceso de preparar una aplicación para su despliegue. Durante el proceso, el código se compila, optimiza y organiza para que tenga un funcionamiento

“eficiente”.

- **Archivos generados:** son los archivos finales que se crean después del Build, en su mayoría suelen ser HTML, CSS y JavaScript. Los archivos se suben al servidor y los usuarios pueden usar la aplicación web.
- Describe despliegue en servidor web.
 - **Despliegue en servidor web:** Consiste en publicar la aplicación para que se pueda acceder a ella a través de Internet o una red normal. Los archivos que se generan después del build se copian al servidor web y los envía a los navegadores de los usuarios cuando acceden a la aplicación.
- Relación con backend Java y APIs.
 - El backend Java usa APIs para **permitir la comunicación** con otras aplicaciones o servicios. Con las APIs se **envían y reciben datos** usando peticiones HTTP. Por ejemplo, un backend Java puede ofrecer una API que se use para una aplicación móvil en lugar de una web.